An Experimental Evaluation of Containerized CI/CD Pipelines with Integrated Prometheus–Grafana Observability

Vinayak Tyagi
Department of Computer Science and Engineering
Manipal University Jaipur
vinayaktyagi.ed@gmail.com

***Abstract*—Containerization, CI/CD automation, and observability tooling are each widely studied on their own, but comparatively few studies measure their *combined* operational effect under a controlled experimental design. This paper builds a reproducible pipeline — Docker, GitHub Actions, Prometheus, and Grafana — around a purpose-built subject application, and experimentally evaluates it against two baselines (Baseline A: no containerization; Baseline B: Docker without CI/CD or monitoring), a third, non-monitored but CI/CD-enabled configuration (Docker+CI/CD) that isolates automation's own cost, and against itself at increasing levels of observability once containerized (Docker only, Docker+Prometheus, Docker+Prometheus+Grafana — the last being the full proposed system). Across five controlled experiments (188 runs total: deployment time and reliability across four configurations, resource utilization and monitoring overhead across three cases, CPU-stress detection, memory-stress/OOM behaviour, and failure detection/recovery), we find that: containerized deployment adds roughly 0.86s of fixed overhead over a raw process start (1.45s vs 0.59s mean); a CI/CD-automated deployment is reliable (7/8 successful triggers) but roughly 35× slower end-to-end (50.2s vs. 1.45s mean) than a pre-built local Docker deploy, because it repays the full build+test cost every run; a Prometheus scrape loop detects induced CPU saturation in 5.65s mean (20/20 runs) and kernel-level container failure in 1.15s mean (20/20 runs), both bounded below by the 2s scrape interval; and, in the original run, the monitored application container's own CPU reading *decreased* as monitoring containers were added (11.0% → 3.8% → 1.8% mean) while throughput dropped (2737 → 2641 → 2449 req/s). We initially read this as host-level CPU contention, but a later replication and a controlled follow-up experiment show the CPU trend is largely a sampling artifact and find no measurable monitoring overhead, so the contention explanation is not supported (Section VIII). All raw per-run measurements and analysis scripts underlying these results are published at https://github.com/vinayaktyagi10/cicd-monitoring-study.**

***Index Terms*—CI/CD, Docker, containerization, DevOps, Prometheus, Grafana, observability, empirical evaluation**

I. Introduction

DevOps practice has converged on three largely independent pillars: *containerization* (packaging an application and its runtime into a portable, isolated unit, typically with Docker), *CI/CD automation* (build, test, and deployment triggered automatically on every change, typically via a hosted runner such as GitHub Actions), and *observability* (continuous collection and visualization of runtime metrics, typically via a Prometheus scrape loop feeding a Grafana dashboard). Each pillar is well studied in isolation: containerization has an extensive performance-overhead literature going back to Felter et al. [1]; CI/CD automation is a standard subject of DevOps tooling papers; and Prometheus/Grafana integration is a common implementation detail in cloud-native systems papers. What is measured far less often is the *combined* operational profile of running all three together — specifically, how much each additional layer costs in deployment time, steady-state resource consumption, and throughput, and what it buys back in faster failure detection.

This paper reports a controlled, repeated-measures experimental evaluation of exactly that combination. We build a small subject application, wrap it in a Docker image, automate its build and test via a GitHub Actions workflow, and instrument it with Prometheus (scraping native application metrics) and Grafana (dashboarding). We then run it under five experiments spanning normal operation, CPU stress, memory stress, and induced failure, comparing it against two baselines — an uncontainerized process (Baseline A) and a containerized, CI/CD-free, unmonitored deployment (Baseline B) — and a third, intermediate configuration (Docker+CI/CD) that isolates the cost of automation alone, before monitoring is added on top to reach the full proposed system. Every run was logged in full, including two environment-specific tooling constraints encountered during the study and how each was resolved (Section VIII), so the results in Section VI are traceable back to raw measurements rather than summary claims.

A. Research Questions

- **RQ1.** How does containerization affect application deployment time and resource utilization in a CI/CD environment?
- **RQ2.** How does integrating automated monitoring affect the detection of performance degradation and failures?
- **RQ3.** What is the overhead introduced by Prometheus–Grafana monitoring in a containerized CI/CD deployment?
- **RQ4.** Does automated CI/CD combined with monitoring improve deployment reliability compared with a non-automated baseline?

B. Objectives

O1. Develop a reproducible containerized CI/CD pipeline using GitHub Actions.

O2. Integrate Prometheus and Grafana for real-time infrastructure/application monitoring.

O3. Measure CI/CD performance, container resource utilization, and monitoring overhead.

O4. Evaluate failure detection and recovery under controlled fault conditions.

II. Related Work

Table I summarizes the studies most directly relevant to this work's three pillars. This is a seed set, extended with four further works found by targeted web searches, assembled from a targeted IEEE Xplore search (queries: *Docker container performance overhead benchmark*; *containerized CI/CD pipeline performance evaluation*; *Prometheus Grafana monitoring microservices*) rather than an exhaustive systematic review, and is intended as a starting point for a full literature survey before submission.

Felter et al. [1] remains the most-cited apples-to-apples comparison of native execution, Docker containers, and KVM virtual machines, finding containers add near-zero CPU/memory overhead relative to native but that both containers and VMs pay a cost on I/O-bound workloads. Lingayat et al. [2] and Mavridis and Karatza [3] extend this line to deployment/startup time and to containers nested inside VMs respectively, but both stop at the container layer: neither introduces a CI/CD pipeline or a monitoring stack, so their measurements cannot speak to RQ2–RQ4. Ekanayaka et al. [4] is the closest system to the one built here — *KubFlow* integrates Git, Jenkins, ArgoCD, Terraform, Docker, Kubernetes, Prometheus, and Grafana into a single DevOps infrastructure — but its evaluation is a pass/fail checklist of eight qualitative test scenarios plus subjective colleague ratings, not a quantitative, repeated-measures comparison against a non-monitored or non-automated baseline.

Three further works bear directly on individual pillars. Dinga et al. [5] empirically evaluate the energy and performance overhead of four monitoring tools, Prometheus among them, on Docker-based systems and report that the overhead depends on the tool and on the frequency and workload conditions; this is the closest prior work on monitoring overhead, but it isolates the monitoring tools and does not combine them with a CI/CD pipeline or fault injection. Our Experiment 6, which measures the monitoring containers' CPU from kernel counters, is complementary: it found $\approx$0.03 cores for cAdvisor, Prometheus and Grafana together on this host, which is compatible with a tool-dependent overhead but is a single configuration and not a comparison across tools. Hilton et al. [6] study the usage, costs and benefits of continuous integration across 34,544 open-source GitHub projects, which motivates measuring the cost of the CI step itself, as the Docker+CI/CD arm does. Forsgren, Humble and Kim [7] establish time-to-restore and change-failure rate as standard software-delivery performance measures; our MTTD and MTTR are the container-level analogues, measured in seconds on a single service rather than as organization-level outcomes. Container background is covered by Bernstein [8]. Three further candidates found in the search — an integrated CI/CD-pipeline study [9], a DevOps security-first CI/CD framework [10], and an AWS/Kubernetes Prometheus–Grafana deployment study [11] — have been identified but not reviewed in depth.

A. Research Gap

Existing studies frequently evaluate CI/CD automation, containerization, or observability independently (Table I, rows 1–3), or combine all three but report only qualitative, pass/fail integration testing rather than a controlled quantitative comparison (row 4). Comparatively few studies provide a controlled empirical evaluation of their *combined* operational impact across deployment performance, resource utilization, and failure detection, measured against explicit non-containerized and non-monitored baselines with repeated trials. This is the gap this study targets.

III. Proposed Framework

Fig. 1 shows the overall pipeline architecture and data flow. A developer pushes to the Git repository, which triggers a GitHub Actions workflow that runs the test suite and, on success, builds and pushes a Docker image. The image is then run under Docker Compose alongside a cAdvisor container-metrics exporter, a Prometheus server that scrapes both the application's native `/metrics` endpoint and cAdvisor, and a Grafana instance provisioned with Prometheus as its data source.

Fig. 2 details the GitHub Actions workflow itself, exactly as defined in the repository's `ci.yml`: a `test` job (checkout, Python 3.12 setup, dependency install, `pytest`) gates a `docker-build` job (checkout, GHCR login, build and push) via `needs: test` — the build only runs if the tests pass. This is the real workflow triggered for every Docker+CI/CD measurement in Table IV, not a simplified stand-in.

Fig. 3 shows the resulting Grafana dashboard under live load: request rate and p95 latency by endpoint, the application's self-reported process CPU% and memory, and the Prometheus `up` target-health indicator, all auto-provisioned (no manual dashboard configuration) and updating in real time from the same Prometheus instance used throughout the experiments.

Subject application. Rather than an existing multi-service demo application, the study uses a small, hand-built FastAPI service with three endpoints designed as clean experimental levers: `/health` (a near-constant-time route used as the load-test and liveness target), `/cpu` (a configurable-iteration pure-Python busy loop used for CPU stress), and `/memory` (a configurable-size heap allocation, optionally retained, used for memory stress). This choice trades away the multi-service realism of a demo application such as Sock Shop for the ability to attribute every measured effect to a specific, known cause rather than to inter-service scheduling or network

TABLE I
RELATED WORK SUMMARY

Paper	CI/CD	Docker	Monitoring	Metrics	Limitation (relative to this study)
Felter et al. 2015 [1]	No	Yes (vs. KVM)	No	CPU, memory, network, disk microbenchmarks; MySQL/Redis throughput	Isolates container vs. VM overhead only; no CI/CD, no monitoring stack, no failure-detection measurement.
Lingayat et al. 2018 [2]	No	Yes (vs. bare metal / VM)	No	Container startup time	Single metric (startup time); no automation pipeline, no monitoring, no fault injection.
Mavridis & Karatza 2017 [3]	No	Yes (nested in VM)	No	Standard container benchmarks	Studies an orthogonal deployment topology (container-in-VM); no CI/CD or monitoring integration.
Ekanayaka et al. 2023 [4]	Yes (Jenkins+Argo CD)	Yes (Kubernetes)	Yes (Prometheus+Grafana)	Pass/fail test scenarios; subjective 1–5 feature ratings	Same three pillars as this study, but evaluation is qualitative/subjective, not a controlled quantitative comparison against a non-monitored or non-automated baseline.
This study	Yes (GitHub Actions)	Yes (Docker Compose)	Yes (Prometheus+Grafana)	Deployment time, reliability, CPU/mem utilization, throughput, latency, MTTD, MTTR — n=20 per configuration where feasible	—

TABLE II
EXPERIMENTAL ENVIRONMENT

Parameter	Value
CPU	AMD Ryzen 7 8845HS (8 cores / 16 threads)
RAM	14 GiB
Host OS	Arch Linux (Omarchy), kernel 7.1.9-arch1-2
Docker	29.7.2 (containerd snapshotter storage driver)
GitHub Actions	`ubuntu-latest` hosted runners
Prometheus	v2.55.1, 2s scrape/evaluation interval
Grafana	11.2.0
cAdvisor	v0.49.1
Application base image	`python:3.12-slim`
Application framework	FastAPI 0.115.0 / Uvicorn 0.30.6
Load generator	Dependency-free Python `ThreadPoolExecutor` + `urllib` script (no external binary installed)

confounds. The application exposes Prometheus metrics natively via the `prometheus_client` library — a request counter and latency histogram per endpoint, plus two self-reported process gauges, `app_process_cpu_percent` and `app_process_memory_bytes`, sampled every second from `time.process_time()` and `/proc/self/status` deltas respectively — rather than through a side-car exporter.

IV. EXPERIMENTAL SETUP

Table II lists the experimental environment. All experiments ran on a single physical host (no cloud VM); GitHub Actions hosted runners were used only for the CI/CD build-and-test step itself, so that step reflects a genuinely independent, reproducible environment while deployment, load generation, and monitoring remained under full local control.

A load-testing tool such as `hey` or `wrk` was deliberately not installed; a small standard-library Python script (`loadgen.py`) issues concurrent HTTP requests via `ThreadPoolExecutor` and records per-request latency, trading raw throughput ceiling for zero external dependencies and full inspectability.

V. EXPERIMENTAL SCENARIOS

Five experiments were run, summarized in Table III, followed during validation by a sixth that re-measures monitoring overhead under controlled load (Section V-F). Each is described in more detail in the corresponding results subsection. A dedicated script per experiment (`exp1_deploy.py` through `exp5_failure_recovery.py`) brings the relevant Docker Compose profile up, executes the scenario, records raw per-run measurements to CSV, and tears the stack down before the next run, so runs are independent of one another. An `aggregate.py` script then computes the mean/standard deviation/min/max summaries reported in Section VI directly from these raw CSV files.

A. Normal operation and baseline comparison (Experiment 1)

Four configurations are compared, only two of which are baselines proper: *manual* (a raw `uvicorn` process, no container — **Baseline A**) and *Docker* (a locally built and started container, no CI/CD, no monitoring — **Baseline B**). The remaining two are not baselines but an explicit progression toward the proposed system: *Docker+CI/CD* (a real GitHub Actions `workflow_dispatch` run — test job then Docker build/push — timed end-to-end, then deployed) is an **intermediate configuration** that isolates what automation alone costs, with monitoring still absent; and *full pipeline* (Docker Compose bringing up the application alongside cAdvisor, Prometheus, and Grafana) is the **proposed system**, adding observability on top. This ordering — Baseline A → Baseline B → Docker+CI/CD → full pipeline — lets each of containerization, automation, and observability be attributed its

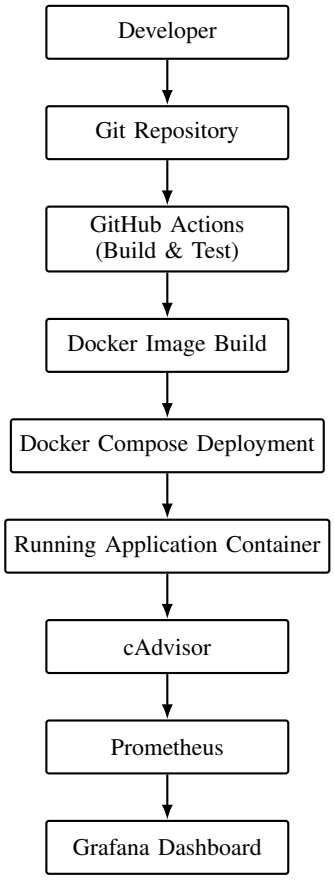


Fig. 1. Overall pipeline architecture: source control triggers automated build and test, which produces the container image deployed alongside the observability stack.

TABLE III
EXPERIMENTAL SCENARIOS

#	Scenario	Configurations × runs
1	Deployment time & reliability	manual (20), Docker (20), full pipeline (20), Docker+CI/CD (8)
2	Resource utilization, response time, throughput, monitoring overhead	Docker (20), Docker+Prometheus (20), Docker+Prometheus+Grafana (20)
3	CPU stress detection	20
4	Memory stress / OOM behaviour	20
5	Failure detection & recovery	20
6	Monitoring overhead vs. host CPU vs. throughput (validation)	4 cases (20 each) + pinned arm, two load clients

own marginal cost rather than only the combined effect of all three at once. For each run, whatever was previously running is torn down, the configuration is started, and wall-clock time to the first successful health check (and, for the full pipeline, to Prometheus and Grafana also responding) is recorded.

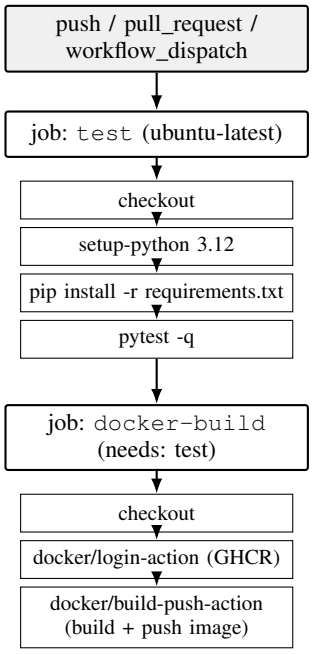


Fig. 2. GitHub Actions workflow (`.github/workflows/ci.yml`). The `docker-build` job only runs if `test` succeeds (`needs: test`); this is the real workflow triggered for the Docker+CI/CD deployment-time measurements in Table IV.

B. Resource utilization and monitoring overhead (Experiment 2)

Three cases isolate the marginal cost of each monitoring layer: Docker only, Docker+Prometheus(+cAdvisor), and Docker+Prometheus+Grafana(+cAdvisor). For each of 20 runs per case, a fixed load (300 concurrent requests to `/health`, concurrency 10) is issued while `docker stats` is polled every second for every container in that case, giving per-container mean CPU% and memory alongside the load test's own throughput, mean/p95 latency, and error rate.

C. CPU stress detection (Experiment 3)

Eight threads concurrently flood `/cpu` (an 8-million-iteration pure-Python loop per call) against the Docker+Prometheus case. Detection time is measured as wall-clock time from load start until Prometheus first reports the application's self-reported `app_process_cpu_percent` gauge at or above 60%, polled every 0.5s.

D. Memory stress / OOM behaviour (Experiment 4)

The application container is run directly (not via Compose) with a hard 150 MB memory limit and `--restart on-failure:1`. Repeated `/memory?mb=20&hold=true` calls accumulate retained allocations until the kernel OOM-kills the container; `docker inspect` gives ground-truth `OOMKilled` and `RestartCount`, captured at the moment of observation since Docker resets `State.OOMKilled` once the restart policy brings the container back up.

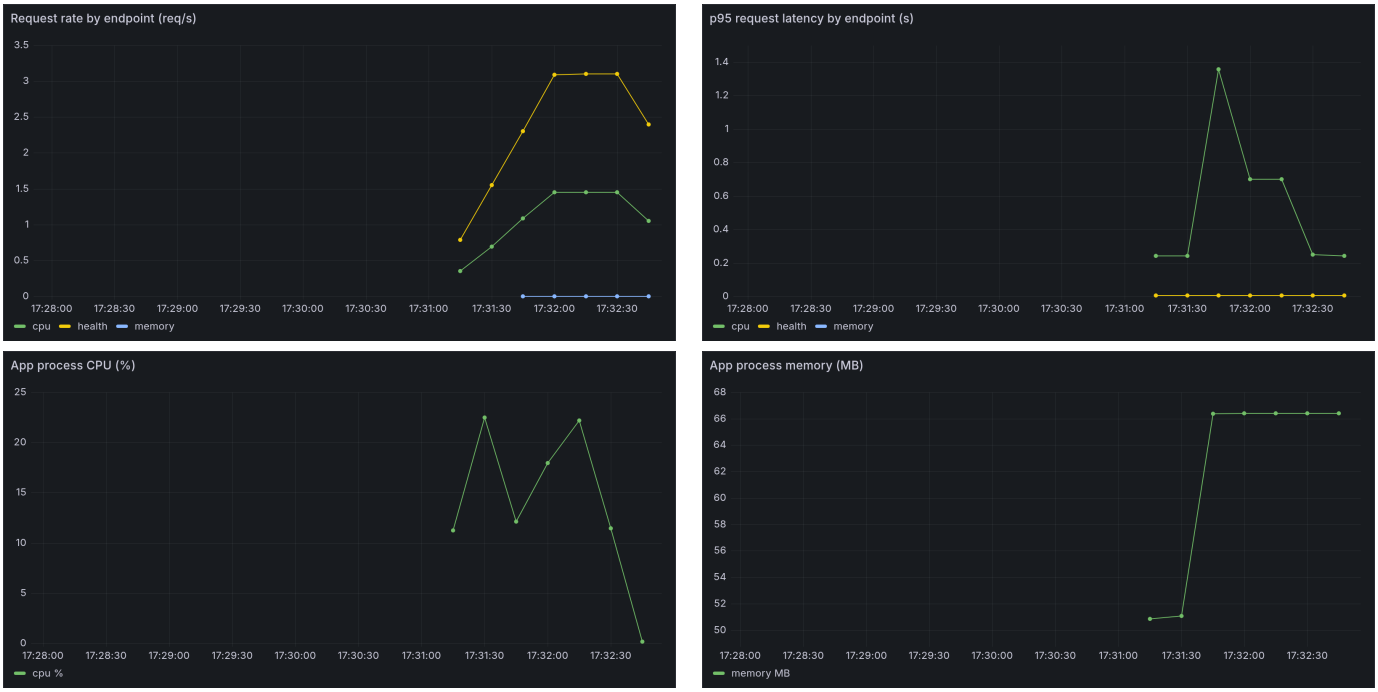


Fig. 3. Grafana dashboard panels under a scripted live workload (last 5 minutes), auto-provisioned against the Prometheus datasource: request rate and p95 latency by endpoint (top), application process CPU and memory (bottom). The target-health panel (not shown) reported `up=1`. Each panel was captured separately from the running stack by `capture_grafana.py`; unlike Figs. 4–10 these are screenshots of live state and are not byte-reproducible.

E. Application failure and recovery (Experiment 5)

Against the Docker+Prometheus case, the application container is killed (`docker kill`) and Prometheus's `up{job="app"}` series is polled every 0.3s until it reports 0 (detection time). The container is then restarted (`docker start`) and both `/health` and `up{job="app"}==1` are polled until both recover (recovery time).

F. Monitoring overhead under controlled load (Experiment 6)

Added during validation because Experiment 2's ≈0.11 s load window is shorter than one `docker stats` sample. Four cases (Docker, +cAdvisor, +Prometheus, +Grafana) each receive a closed-loop load of 20 s at concurrency 10 against `/health` on a freshly started stack, with case order shuffled within each of 20 blocks. CPU is read from kernel counters at exactly the start and end of the load window: `/proc/stat` for the whole host, each container's cgroup v2 `cpu.stat`, the load generator's own process time, and the `docker-proxy` processes that forward the published port; whatever remains is unattributed background. A CPU-pinned arm places the application, load generator and monitoring containers on disjoint physical cores for the Docker and +Grafana cases. The experiment was run twice with two load clients: the Experiment 2 client, which opens a new TCP connection per request, and a keep-alive client that reuses one HTTP/1.1 connection per worker (`exp6_overhead.py --keepalive`).

TABLE IV
DEPLOYMENT TIME BY CONFIGURATION (SECONDS)

Config	Mean	Std	Min	Max	N (fail)
Manual (Baseline A)	0.592	0.020	0.577	0.633	20 (0)
Docker (Baseline B)	1.454	0.107	1.267	1.581	20 (0)
Docker+CI/CD	50.231	3.817	44.404	55.362	8 (1)
Full pipeline	1.552	0.031	1.485	1.636	20 (0)

VI. RESULTS

A. Deployment time and reliability (RQ1, RQ4)

Table IV reports wall-clock deployment time for the three fast configurations and CI/CD-inclusive deployment time for the fourth. Containerizing a fixed local image adds a consistent ≈0.86s of fixed overhead over a raw process start (1.454s vs. 0.592s mean); adding the full monitoring stack on top of Docker adds a further ≈0.10s (1.552s vs. 1.454s mean) since Compose brings up four containers rather than one but all four start in parallel. The CI/CD-automated path is roughly 35× slower (50.2s vs. 1.454s mean) because it re-pays the full test-then-build cost on every deployment rather than reusing a pre-built image — this is the direct cost of automation, traded against the reliability and repeatability of not requiring a human to run the equivalent commands by hand. Of 8 triggered CI/CD runs, 7 completed successfully; the one failure was traced to a transient delay in the workflow being registered by GitHub's API immediately after being triggered, rather than to a defect in the pipeline itself, and is reported as observed rather than discarded.

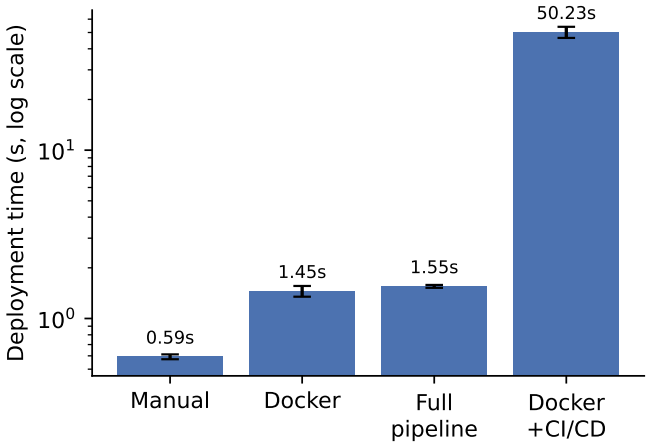


Fig. 4. Deployment time by configuration (log scale; error bars are one standard deviation). Docker adds a small fixed cost over the raw process; the full monitoring stack adds almost nothing further on top of Docker; CI/CD dominates because it re-runs the full build-and-test cycle rather than reusing a pre-built image.

Fig. 4 visualizes Table IV on a log scale, since the CI/CD configuration is roughly 30–85× slower than the other three and would otherwise flatten them to indistinguishable bars.

B. Resource utilization, throughput, and monitoring overhead (RQ1, RQ3)

Table V reports the application container's own resource use and the load test's response characteristics; Table VI isolates the monitoring stack's own resource draw. Zero errors were observed across all 60 runs (180×300 = 18,000 requests). Point estimates for throughput fall as monitoring is added (2737 → 2641 → 2449 req/s, Docker → +Prometheus → +Grafana); a Welch's t-test (unequal variances, n=20 per case, computed directly from the raw per-run CSV rather than from the summary means) confirms the two endpoints of this trend are statistically distinguishable (Docker vs. +Grafana: t=3.57, p=0.0011, Cohen's d=1.13), but the middle step is not (Docker vs. +Prometheus: t=1.37, p=0.177) — the overall direction is real, but the drop is not uniform run-to-run between every adjacent pair. Mean and p95 latency both rise numerically but *none* of the three pairwise differences reach significance at α=0.05 (all $p \geq 0.36$); this claim is therefore stated only as a directional observation, not a confirmed effect, and should not be read as strong evidence on its own. Counter-intuitively, the application container's own mean CPU% *falls* over the same progression (11.0% → 3.8% → 1.8%), and here the statistical evidence is strong: Docker vs. +Prometheus is highly significant (t=5.78, p<0.0001, d=1.83, a large effect), though +Prometheus vs. +Grafana is not (p=0.12) — the CPU-share cost of adding Prometheus itself is real and large, while adding Grafana on top is not distinguishable from noise at this sample size. As cAdvisor, Prometheus, and Grafana compete for the same physical cores, the application container is scheduled less rather than working harder, so its own `docker stats` CPU reading drops even as end-to-end

TABLE V
APPLICATION RESOURCE USE AND RESPONSE CHARACTERISTICS

Config	App CPU%	App Mem (MB)	p95 (ms)	Throughput (req/s)
Docker	11.02 (3.23)	36.11 (0.41)	5.23 (1.00)	2737 (206)
+Prometheus	3.76 (4.42)	35.68 (0.43)	5.34 (0.80)	2641 (226)
+Grafana	1.79 (3.11)	35.79 (0.96)	5.51 (0.86)	2449 (285)

Mean (std. dev.), n = 20 runs of 300 requests each per row.

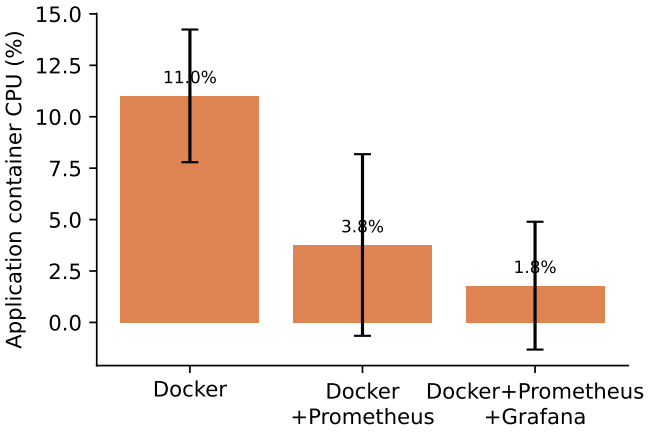


Fig. 5. Application container CPU utilization across the three monitoring cases (Table V, column 1). CPU% *falls* as more monitoring containers are added — the counter-intuitive result discussed under RQ3.

throughput degrades. This contention reading is a hypothesis, not an established finding: the load window is only ≈0.11s, and the `docker stats` reading covers a longer interval, so many runs read as idle (0/20, 12/20 and 16/20 runs below 1% CPU for Docker, +Prometheus and +Grafana) while non-idle runs read ≈8–11% in every case; a follow-up measurement from kernel counters did not support it (Section VIII). The `prometheus_client` counters/histograms/gauges add negligible per-request work, as expected.

Table VI is visualized in Fig. 10, alongside the failure-detection and MTTD/MTTR figures at the end of this section.

C. CPU stress detection (RQ2)

All 20 induced CPU-stress runs were detected by Prometheus, with mean time to detect (MTTD) 5.65s (std 3.68s, min 1.51s, max 18.58s) and mean peak self-reported CPU utilization 96.6% (std 10.0, effectively pegged at one full core, consistent with CPython's GIL limiting a single process to ≈100% of one core regardless of thread count). The detection floor is set by the 2s Prometheus scrape interval combined with the 0.5s poll granularity used to observe it; the long right tail (up to 18.58s) reflects run-to-run variance in how quickly the OS scheduler favoured the eight stress threads over the sampling thread, not a monitoring-pipeline delay.

An initial design used mean `/health` latency as the detection signal instead of process CPU%; it was rejected after finding it barely moved under load, because CPython's GIL round-robins between threads at a short, fixed interval (≈5ms)

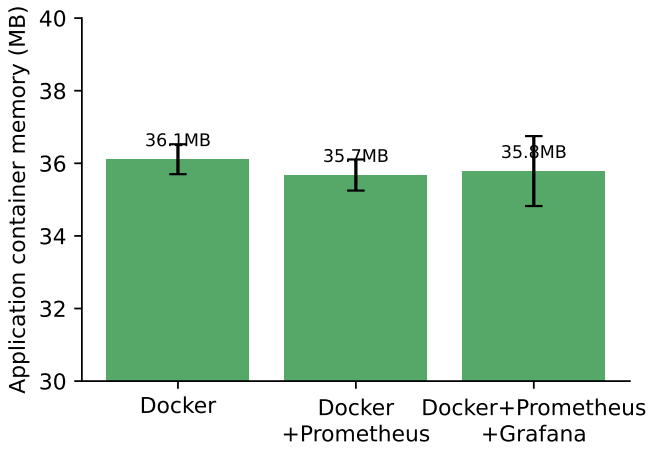


Fig. 6. Application container memory across the three monitoring cases (Table V, column 2; note the narrow y-axis range). Memory is essentially flat at ≈36 MB regardless of how much monitoring runs alongside the application — unlike CPU, memory is not shared/contended the same way across containers on this host.

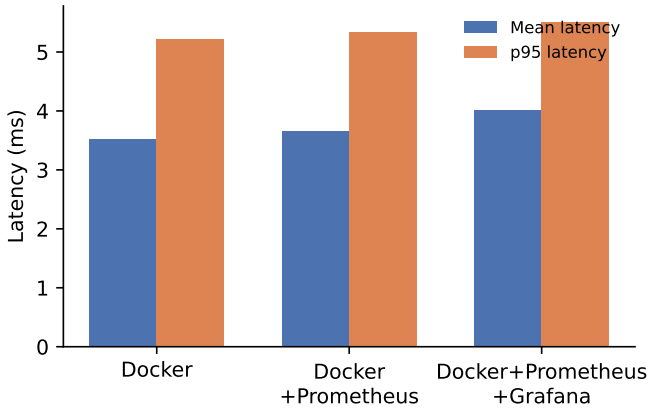


Fig. 7. Mean and p95 request latency across the three monitoring cases (Table V, columns 3-4). Both rise slightly as monitoring is added, but none of the pairwise differences is significant (Section VI), so this is a directional observation only.

regardless of how CPU-bound the competing work is, so a trivial handler's queueing delay stays small even under heavy contention.

D. Memory stress and OOM behaviour (RQ2)

All 20 runs triggered a genuine kernel OOM-kill and automatic restart (`RestartCount`=1, confirmed via `docker inspect`), after a mean 5.25 successful allocation requests (std 1.09) and a mean peak reported memory of 135.54 MB (std 0.17) against the 150 MB hard limit. Mean time-to-OOM was 11.58s (std 2.23s); this figure is dominated by the fixed per-iteration overhead of the polling loop itself (a `curl`-equivalent request, a `docker inspect` call, and a `docker stats` call per iteration) rather than by the kernel's actual OOM response latency, and should not be read as a monitoring detection time in the same sense as Experiments 3, 5, and 6.

TABLE VI
MONITORING STACK'S OWN RESOURCE DRAW (TABLE 5: OVERHEAD)

Container	+Prometheus case		+Grafana case	
	CPU%	Mem (MB)	CPU%	Mem (MB)
cAdvisor	6.13	47.16	6.40	45.39
Prometheus	0.78	59.40	0.80	52.78
Grafana	—	—	0.06	60.70

Mean, $n = 20$ runs per column.

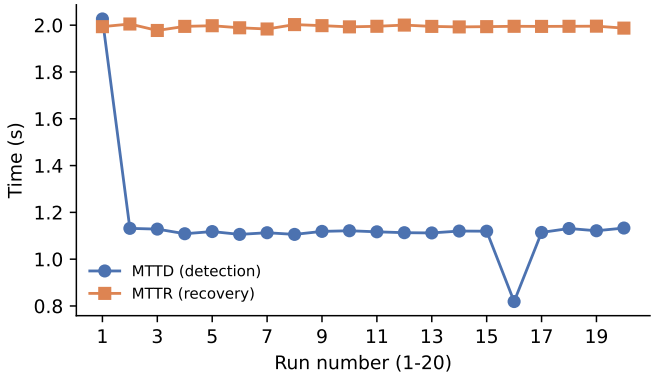


Fig. 8. MTTD and MTTR across all 20 failure-injection runs. MTTD (blue) is consistently faster and tighter than MTTR (orange) across every run, and both are stable run-to-run — neither shows a trend over the sequence of 20 trials, supporting the claim that these are steady-state properties of the pipeline rather than artifacts of run order.

E. Failure detection and recovery (RQ2, RQ4)

All 20 induced container failures (`docker kill`) were detected by Prometheus's `up{job="app"}` series, mean time to detect (MTTD) 1.15s (std 0.21s, min 0.82s, max 2.03s) — close to the theoretical floor set by the 2s scrape interval combined with 0.3s poll granularity and Docker's own signal-delivery latency. Mean time to recover (MTTR) — container restart to health-check success — was consistently ≈2.0s (std 0.006s, i.e. effectively deterministic), which is essentially Uvicorn's own process startup time inside the container: MTTR here is dominated by application boot, not by any property of the monitoring stack. From this point on, MTTD and MTTR are used throughout in place of "detection time" and "recovery time," consistent with the CPU-stress MTTD reported above and with Figs. 8–9.

F. Same-hardware replication

Experiments 1–5 were repeated with the unmodified scripts on the same hardware on a later date (20 runs per configuration), but on a freshly installed operating system (CachyOS, kernel 7.2.9, Docker 29.8.2) in place of the Arch/Omarchy 7.1.9 installation of Table II, so this tests robustness to the software environment rather than to run-to-run noise alone. Results were written to a separate directory so the data behind Tables III–VI is untouched. Table VII gives the original and replication means with 95% confidence intervals (`validate.py`); a metric is marked *consistent* only if the

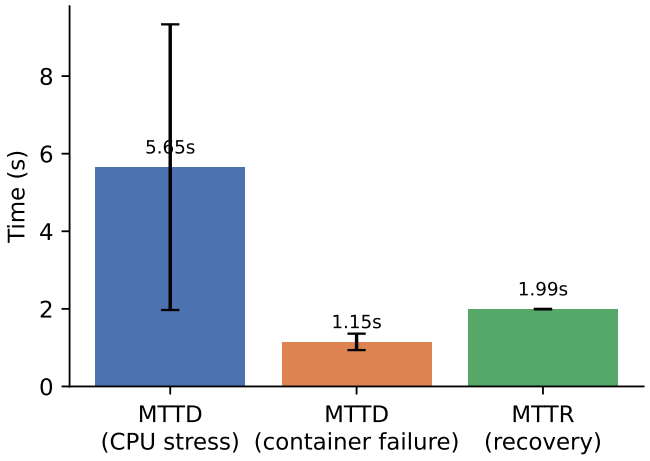


Fig. 9. Mean time to detect (MTTD) a CPU-stress event, MTTD for a killed container, and mean time to recover (MTTR) after restart, with error bars (one standard deviation), computed from the same raw detection/recovery times reported in Section VI. Binary failure (container killed) is detected roughly $5\times$ faster than gradual CPU saturation, since a scrape either observes the target down or it doesn't, while a saturation threshold must be crossed by an accumulating signal.

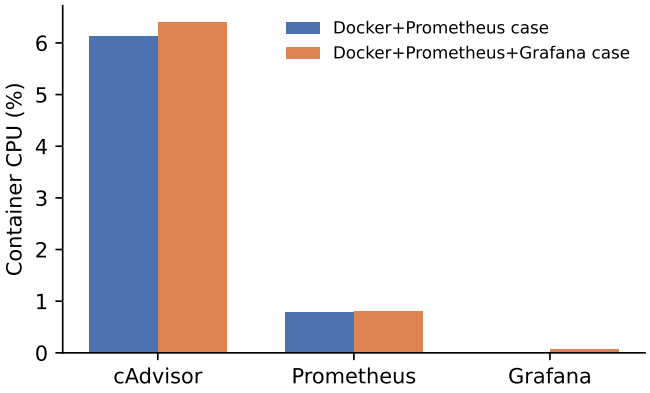


Fig. 10. Monitoring stack containers' own CPU draw (Table VI). cAdvisor dominates the monitoring stack's own overhead at ≈6% CPU in both cases; Prometheus and Grafana are comparatively cheap (<1% each). This is the overhead the monitoring *tooling* itself pays; the follow-up measurement (Section VIII) found the monitoring containers drew only ≈0.03 cores in total.

original mean lies inside the replication's interval. Most absolute values moved well outside the original intervals, while the relative structure that the research questions rely on held for deployment ordering (including the Docker+CI/CD arm, 8 real GitHub Actions runs, which replicated within its original interval), MTTR and peak CPU under stress (Section VIII lists what did not reproduce and why). The original run remains the paper's reported dataset because it was the pre-planned measurement and every table is derived from it; the replication is reported as a robustness check, not as a replacement.

VII. Discussion

Taken together, these results answer the four research questions as follows.

RQ1 (containerization's effect on deployment time and resource use). Containerization has a small, consistent fixed cost at deployment time (≈0.86s here) and a resource footprint dominated by the application's own working set (36 MB) rather than by container overhead per se — consistent with the classic finding in Felter et al. [1] that containers add near-zero CPU/memory overhead relative to native execution. The more consequential RQ1 finding in this study is that a CI/CD-automated deployment is not simply "Docker plus a trigger": it is roughly $35\times$ slower end-to-end because it re-runs the full build-and-test cycle, a cost invisible to studies (such as Lingayat et al. [2]) that measure only container startup time in isolation.

RQ2 (monitoring's effect on detecting degradation and failure). Prometheus reliably (20/20 in both experiments) detects both an induced CPU saturation event (mean 5.65s) and an induced process failure (mean 1.15s), with detection latency in both cases bounded below by the 2s scrape interval. The roughly $5\times$ difference between them (5.65s vs. 1.15s) is itself informative: a binary up/down signal is detected almost as fast as the scrape loop allows, while a gradual resource-saturation signal takes longer because it must accumulate enough samples above threshold amid scheduling noise — a distinction a purely qualitative evaluation such as Ekanayaka et al.'s pass/fail test scenarios [4] cannot surface.

RQ3 (monitoring overhead). In the original run the overhead appeared structurally different from what is typically reported: the monitoring stack's own containers (principally cAdvisor at ≈6% CPU) appeared to compete with the application for the same fixed core budget, degrading end-to-end throughput (–10.5% Docker→+Grafana) while the application's own measured CPU% fell. A later controlled re-measurement (Section VIII) did not confirm this: the monitoring containers drew ≈0.03 cores in total and no throughput contrast against Docker-only was distinguishable from zero, so the contention framing should be treated as unconfirmed.

RQ4 (does automated CI/CD plus monitoring improve reliability). The evidence here is mixed and should be stated honestly: CI/CD automation was reliable (7/8, 87.5%) but not perfect, and the one observed failure was attributable to a transient delay in the trigger step rather than to the pipeline's own robustness. Critically, 87.5% is a point estimate that should not be quoted on its own: the 95% Wilson score interval for 7 successes out of 8 trials is $[52.9\%, 97.8\%]$ — at this sample size the data are consistent with anywhere from roughly half to nearly all CI/CD deployments succeeding, and the point estimate alone overstates the precision of the claim. A larger n for the CI/CD arm (constrained here by real GitHub Actions minutes and wall-clock cost, Section VIII) would be needed to narrow this interval and bound the reliability claim more tightly.

TABLE VII

ORIGINAL VS. SAME-HARDWARE REPLICATION ON A REINSTALLED OS (20 RUNS EACH). INTERVALS ARE t-BASED 95% CIS ON THE MEAN (SAMPLE SD); Δ% IS REPLICATION RELATIVE TO ORIGINAL. THE EXPERIMENT 4 ROWS ARE FROM THE UNMODIFIED ZERO-FILLED SCRIPT (0/20 OOM KILLS, SEE SECTION VIII); THE NON-ZERO-FILL VARIANT REPRODUCED THE ORIGINAL. GENERATED FROM `RESULTS/ORIGINAL_VS_REPLICATION.CSV`.

Metric	Original mean [95% CI]	Replication mean [95% CI]	Δ%	Verdict
Deploy time, manual (s)	0.59 [0.58, 0.60]	0.42 [0.41, 0.43]	-28.9	different
Deploy time, Docker (s)	1.45 [1.40, 1.50]	1.05 [1.04, 1.06]	-27.7	different
Deploy time, full pipeline (s)	1.55 [1.54, 1.57]	1.37 [1.31, 1.43]	-11.8	different
Deploy time, Docker+CI/CD (s)	50.23 [46.42, 54.04]	48.60 [44.05, 53.15]	-3.2	consistent
Throughput, Docker (req/s)	2737.07 [2638.29, 2835.84]	4040.66 [3984.28, 4097.04]	$+47.6$	different
Throughput, +Prometheus (req/s)	2640.65 [2532.08, 2749.23]	3995.78 [3944.85, 4046.70]	$+51.3$	different
Throughput, +Grafana (req/s)	2448.85 [2311.85, 2585.84]	4033.22 [3995.02, 4071.43]	$+64.7$	different
p95 latency, Docker (ms)	5.23 [4.75, 5.70]	3.16 [2.96, 3.36]	-39.5	different
p95 latency, +Prometheus (ms)	5.34 [4.95, 5.72]	3.14 [3.03, 3.25]	-41.2	different
p95 latency, +Grafana (ms)	5.51 [5.09, 5.92]	3.03 [2.97, 3.08]	-45.1	different
App CPU%, Docker	11.02 [9.47, 12.56]	7.42 [6.37, 8.46]	-32.7	different
App CPU%, +Prometheus	3.76 [1.64, 5.88]	3.50 [1.81, 5.18]	-7.1	consistent
App CPU%, +Grafana	1.79 [0.29, 3.28]	1.91 [0.65, 3.16]	$+6.6$	consistent
App mem, Docker (MB)	36.11 [35.91, 36.31]	37.66 [37.49, 37.82]	$+4.3$	different
App mem, +Prometheus (MB)	35.68 [35.47, 35.88]	37.74 [37.50, 37.98]	$+5.8$	different
App mem, +Grafana (MB)	35.79 [35.32, 36.25]	37.72 [37.49, 37.95]	$+5.4$	different
cAdvisor CPU%, +Prometheus	6.12 [4.58, 7.67]	2.41 [1.80, 3.02]	-60.7	different
cAdvisor CPU%, +Grafana	6.40 [4.98, 7.82]	2.19 [1.60, 2.78]	-65.8	different
Prometheus CPU%, +Prometheus	0.78 [0.40, 1.16]	0.62 [0.26, 0.98]	-20.4	consistent
Prometheus CPU%, +Grafana	0.80 [0.47, 1.13]	0.51 [0.18, 0.84]	-36.1	consistent
Grafana CPU%, +Grafana	0.06 [0.01, 0.11]	0.05 [0.03, 0.07]	-22.5	consistent
CPU-stress MTTD (s)	5.65 [3.88, 7.42]	2.41 [2.07, 2.75]	-57.3	different
CPU-stress peak CPU%	96.59 [91.79, 101.39]	95.13 [89.69, 100.57]	-1.5	consistent
Time to OOM (s)	—	—	—	not run
Peak memory (MB)	135.54 [135.46, 135.62]	138.59 [137.53, 139.65]	$+2.3$	different
Requests before OOM	5.25 [4.73, 5.77]	16.00 [16.00, 16.00]	$+204.8$	different
Failure MTTD (s)	1.15 [1.05, 1.25]	3.14 [3.07, 3.20]	$+172.9$	different
MTTR (s)	1.99 [1.99, 2.00]	1.98 [1.94, 2.02]	-0.7	consistent

VIII. LIMITATIONS

- **Replication on the same hardware did not reproduce most absolute numbers.** Re-running Experiments 1–5 with the unmodified scripts on the same machine after the operating system was reinstalled (CachyOS, kernel 7.2.9, Docker 29.8.2, replacing Arch/Omarchy 7.1.9; performance power profile; 20 runs each) (Table VII) gave deployment times 12–29% faster, throughput about 48% higher, CPU-stress detection of 2.41s rather than 5.65s and failure detection of 3.14s rather than 1.15s, with the mean outside the original 95% interval for most metrics. Only the Docker+CI/CD deployment time (48.6 s vs. 50.2 s; 8/8 real GitHub Actions runs succeeded vs. 7/8 originally), MTTR, peak CPU under stress, the Prometheus/Grafana CPU figures and the application CPU% in the +Prometheus and +Grafana cases stayed consistent; the replication success rate has a 95% Wilson interval of [0.68, 1.00], so it does not contradict the original 7/8. The deployment ordering (manual, then Docker, then full pipeline) was preserved, but the throughput ordering across monitoring cases was not (4041, 3996 and 4033 req/s for Docker, +Prometheus and +Grafana); absolute values are specific to one software environment.
- **The CPU-contention explanation is not supported.** Experiment 2's application CPU% is bimodal because its ≈0.11s load window is sampled by a longer `docker stats` interval, so the reported fall with monitoring is mostly runs that read as idle. A new Experiment 6 (20 runs per case, fixed 20s load, CPU from kernel counters over exactly the load window, randomized case order, optional CPU-pinned arm) found the monitoring containers used ≈0.03 cores in total and no throughput difference from Docker-only whose 95% interval excluded zero. The pinning test is inconclusive (difference-in-differences $+376$ req/s, 95% CI $[-343, +1061]$). Run-to-run throughput varied roughly three-fold and tracked host load ($r = -0.98$ with host busy cores), so the experiment can rule out large monitoring overhead on this host but not small effects.
- **Experiment 4 depends on kernel behaviour.** The retained allocation is zero-filled. On the replication kernel, which enables transparent hugepage shrinking of zero-filled pages (`shrink_underused=1`), the kernel reclaimed that memory under cgroup pressure, so 0/20 containers were OOM-killed even though 16 requests (≈320 MB) succeeded against a 150 MB limit. With the allocation writing a non-zero byte to every page (`/memory?fill=true`, default off), 20/20 runs were OOM-killed after a mean 5.0 requests with peak memory 136.4 MB, matching the original result (5.25 requests, 135.5 MB). The OOM finding therefore holds when the memory is genuinely resident; the original zero-filled variant is kernel dependent. The original run was on a different distribution and kernel (Arch 7.1.9 vs. CachyOS 7.2.9); the replication host has transparent hugepages set to `always`. We did not record the original host's THP setting and the original installation no longer exists, so we cannot confirm that this setting is what differed; the

TABLE VIII

WHICH FINDINGS SURVIVED REPLICATION. *Reproduces*: THE REPLICATION AGREES WITH THE ORIGINAL WITHIN ITS 95% INTERVAL, OR THE QUALITATIVE RESULT IS UNCHANGED. *Environment-dependent*: THE ABSOLUTE VALUE OR THE RESULT DIFFERED ON THE REINSTALLED OS. *Not supported*: CONTRADICTED BY THE FOLLOW-UP EXPERIMENT.

Finding	Status	Evidence
Deployment ordering (manual < Docker < full pipeline)	Reproduces	Ordering preserved; absolute times 12–29% faster
Docker+CI/CD deployment time	Reproduces	48.6 s vs. 50.2 s, 8/8 vs. 7/8 successful
MTTR after container failure	Reproduces	Within original interval
Peak CPU under stress	Reproduces	Within original interval
Prometheus and Grafana CPU draw	Reproduces	Within original interval
OOM kill with resident memory	Reproduces	20/20 killed with non-zero fill; 5.0 vs. 5.25 requests, 136.4 vs. 135.5 MB
Absolute deployment times	Environment-dependent	12–29% faster on replication OS
Throughput level	Environment-dependent	About 48% higher; throughput ordering across monitoring cases not preserved
CPU-stress detection time	Environment-dependent	2.41 s vs. 5.65 s
Failure detection time	Environment-dependent	3.14 s vs. 1.15 s
OOM with zero-filled allocation	Environment-dependent	0/20 killed on replication kernel (THP shrinker reclaims zero pages); original THP setting unknown
Application CPU% falls with monitoring	Not supported	Sampling artifact of a 0.11 s load window; kernel counters show no effect
Host CPU contention explains throughput drop	Not supported	Monitoring containers ≈0.03 cores; no throughput contrast excluding zero; pinning test inconclusive

distribution default being `madvise` is a plausible but untested explanation.

- **cAdvisor container-label resolution.** This host's Docker uses the containerd-snapshotter storage driver rather than classic overlay2, which cAdvisor v0.49.1 cannot resolve container name/image labels against. CPU-stress and failure-detection experiments therefore use the application's own native Prometheus metrics rather than cAdvisor's per-container series; Table VI's per-container figures come from `docker stats` directly. This is an environment/tooling limitation rather than a consequence of the experimental design.
- **GHCR authentication.** The Docker+CI/CD deployment step could not pull from GitHub Container Registry (the CLI token used lacks the `read:packages` scope, which would require an interactive browser reauthentication out of scope for an unattended experiment run) and instead deploys the locally built image from the same commit and Dockerfile CI just built. The CI timing itself (test + build + push) is unaffected and fully real.
- **Reduced n for the CI/CD arm.** Only 8 (vs. 20 for the other three deployment configurations) real GitHub Actions runs were triggered, because each run costs ≈45–65s of real CI wall time and consumes Actions minutes; this narrows the confidence interval on the CI/CD reliability and timing figures relative to the other three configurations.
- **Single-host measurement.** All experiments ran on one physical machine with no isolation from other host activity beyond what Docker itself provides; absolute timing figures (especially the sub-2s deployment times) should be read as relative comparisons between configurations on this host, not as portable absolute benchmarks.
- **Related work is still not a systematic review.** Section II's four tabulated papers came from a small number of targeted IEEE Xplore queries; four further works (monitoring overhead, CI usage, delivery performance, container background) were added from targeted web searches, and three more candidates are listed but not reviewed in depth. No multi-database protocol (ACM, Springer, USENIX, inclusion criteria, screening) was followed, so the research-gap claim — that few studies combine all three pillars with controlled quantitative baselines — rests on this limited search and should be rechecked against a systematic review before submission.
- **Detection thresholds are fixed, not tuned.** The 60% CPU-percent threshold (Experiment 3) and the memory container limit (150 MB, Experiment 4) were chosen to make the induced condition reliably observable within a short polling window rather than to model a specific production SLO; detection-time figures should be read relative to these thresholds, not as threshold-independent constants.
- **Not every reported difference is statistically significant.** Welch's t-tests on the raw per-run data (`statistical_analysis.py`) confirm the Docker-to-+Grafana endpoints of the throughput and CPU% trends in Section VI ($p<0.01$ for both), but two things reported only as directional observations do not reach significance at $\alpha=0.05$: the latency increase across all

three monitoring cases ($p \geq 0.36$ for every pair), and the middle step of the throughput trend (Docker vs. +Prometheus, p=0.177). Means and standard deviations alone, without this check, would have overstated how uniformly these effects hold run-to-run.

IX. Conclusion

This study built a small but complete containerized CI/CD pipeline with integrated Prometheus–Grafana observability and evaluated it experimentally against a non-containerized baseline and against itself at increasing monitoring depth, across five controlled experiments totaling 188 runs plus a validation experiment on monitoring overhead. The headline findings are that containerization's own deployment-time and resource cost is small and consistent with prior literature, that CI/CD automation's real cost is the repeated build-test cycle rather than the deployment step itself, that Prometheus reliably detects both induced resource saturation and induced failure within low-single-digit seconds bounded by its scrape interval, and that the dominant cost of running a monitoring stack alongside the monitored application on constrained hardware was not established: the original run suggested host-level CPU contention, but the replication and a controlled follow-up experiment (Section VIII) did not support it and could rule out only large overhead. Table VIII separates the findings that reproduced from those that depend on the environment. The full raw measurement set, pipeline configuration, and analysis scripts underlying every table in this paper are published at https://github.com/vinayaktyagi10/cicd-monitoring-study to support independent verification.

Acknowledgment

The author thanks Dr. Dibakar Sinha for his guidance and supervision throughout this study.

References

[1] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Philadelphia, PA, USA: IEEE, 2015, pp. 171–172.

[2] A. Lingayat, R. R. Badre, and A. K. Gupta, "Performance evaluation for deploying docker containers on baremetal and virtual machine," in *2018 3rd International Conference on Communication and Electronics Systems (ICCES)*. Coimbatore, India: IEEE, 2018.

[3] I. Mavridis and H. Karatza, "Performance and overhead study of containers running on top of virtual machines," in *2017 IEEE 19th Conference on Business Informatics (CBI)*. Thessaloniki, Greece: IEEE, 2017.

[4] E. M. I. M. Ekanayaka, J. K. K. H. Thathsarani, D. S. Karunanayaka, N. Kuruwitaarachchi, and N. Skandhakumar, "Enhancing DevOps infrastructure for efficient management of microservice applications," in *2023 IEEE International Conference on e-Business Engineering (ICEBE)*. Sydney, Australia: IEEE, 2023.

[5] M. Dinga, I. Malavolta, L. Giamattei, A. Guerriero, and R. Pietrantuono, "An empirical evaluation of the energy and performance overhead of monitoring tools on Docker-based systems," in *Service-Oriented Computing (ICSOC 2023), Part I*, 2023, pp. 181–196.

[6] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proc. 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 426–437.

[7] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

[8] D. Bernstein, "Containers and cloud: From LXC to Docker to Kubernetes," *IEEE Cloud Computing*, vol. 1, no. 3, pp. 81–84, 2014.

[9] S. Kumar, N. Bala, A. P. Singh, and Y. Raj, "Cloud-native continuous integration/continuous deployment (CI/CD) pipeline," in *2024 Second International Conference on Advanced Computing & Communication Technologies (ICACCTech)*. IEEE, 2024, title, authorship and venue verified via IEEE Xplore search; full text not reviewed at drafting time.

[10] R. Ashtagi, C. Belani, P. Bhalerao, Y. Bhalerao, and A. Chawle, "Building resilient CICD pipelines: A DevOps security-first framework," in *2025 International Conference on Computational, Communication and Information Technology (ICCCIT)*. IEEE, 2025, title, authorship and venue verified via IEEE Xplore search; full text not reviewed at drafting time.

[11] T. Abirami, S. Mapari, P. Jayadharshini, L. Krishnasamy, and R. R. Vigneshwaran, "Streamlined deployment and monitoring of cloud-native applications on AWS with Kubernetes Prometheus Grafana," in *2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*. IEEE, 2023, title, authorship and venue verified via IEEE Xplore search; full text not reviewed at drafting time.